



Coding Problem Query

...I am using the **navigation stack (nav2)** without problems, but I would like to understand how I can **make the robot stop** from its path if it **finds a dynamic obstacle in front of it** (at a minimum distance of n meters)? Is there a demo or tutorial to take inspiration from?

Not Retrieved $S(Q, D) = 0.31$

Original Document

VelocityPolygonStop

```
action_type: "stop"
min_points: 6
polygon_pub_topic:
"velocity_polygon_stop"
velocity_polygons:      ["rotation",
"translation_forward",
"translation_backward", "stopped"]
points: "[[0.3, 0.3], [0.3, -0.3], [-0.3,
-0.3], [-0.3, 0.3]]"
linear_min: 0.0
translation_forward...
```



RL-Index

$S(Q, \tilde{D}) = 0.55$ Retrieved

Reasoned Document

Key Points:

VelocityPolygonStop: A velocity polygon stop action with a minimum of 6 points is defined...

Explanation: If you're working with motion planning in robotics...**need to define a stop condition where an object comes to a complete halt.** These polygons define the **limits and range for forward translation** for robot to **detect an obstacle in front** and stop at a specific **distance boundary**.



Reasoning Trace



How to make a robot stop if it sees a dynamic obstacle n meters away?



Mismatch between raw configuration code-based original document and language-based query.



The reasoned document connects the user's phrase ("stop at a specific distance") to the technical implementation (defining "polygons" that set limits for forward translation).

Answer Generation

Original document: It just provides a list of coordinates and settings but does not explain why these numbers matter.

Reasoned Document: It explicitly connects the code to the user's goal by using terms like **"stop condition"** and **"complete halt."** It explains that the polygons are the "boundaries" that trigger the stop.

Alignment: The reasoned document empowers the user to follow the Gold Answer's tutorial and successfully adjust the points to match their specific "n meters" safety requirement.



Gold Answer

You can use the **Collision Monitor package inside the Nav2 Stack**...to make the **robot stop**. Here is a tutorial: [Using Collision Monitor]... and here are the sources [Github].